CLAUDE CODE SKILLS

用這 3 層架構
讓 AI 真正幫你工作

Skill 概念・解剖結構・觸發方式
建立流程・Debug 清單・完整速查

CHANNEL

JayLuxAI | AI 自動化

CONTENTS

目錄

什麼是 Skill，為什麼你一定要用 3

Skill 解剖：Frontmatter + 指令 4

支援檔案架構 5

觸發方式：Slash Command vs 自然語言 6

Progressive Context Loading：3 層架構 7

從零建立 Skill：Skill Builder 問答 8

Feedback Cycle：邊用邊優化 9

Debug 清單：6 種症狀對照表 10

Local vs Global Skill 11

三個核心重點 12

資源與社群 13

CORE CONCEPT

什麼是 Skill，為什麼你一定要用

DEFINITION

Skill 就是你給 Claude Code 的一份可重複使用的指令。把一個流程寫清楚，存成 skill，之後一個指令就跑，結果越來越穩定。
最簡單的比喻：**Skill = AI 的 SOP**（標準作業流程）。

三大理由

01 · PERSONAL

個人效率

同時跑 4 個 agent，不是 10% 提升，是 10 倍。

02 · TEAM

團隊槓桿

SOP 數位化，整個組織一起升級。

03 · TIMING

好時機

Skill 生態系剛起步，現在學走在前面。

Skill 能做的事

- 執行多步驟流程（腳本生成、資料分析、報告整理）
- 呼叫 API、執行 script、建立檔案
- 啟動 sub-agent，讓多個 agent 並行工作
- 跨專案被呼叫（global skill）

ANATOMY

Skill 解剖：Frontmatter + 指令

一個 skill 就是一個 `SKILL.md` 檔案，分為兩個部分。

SKILL.MD 結構

PART 1 · FRONTMATTER（用 --- 包起來）

name: transcript

description: *Use when Jay provides a video outline and asks for a script...*

PART 2 · INSTRUCTIONS（步驟指令）

Step 1 讀 patterns.md，套用風格規則

Step 2 根據大綱生成完整腳本

Step 3 存檔，auto-trigger Phase 1.5

Frontmatter 的作用

- 讓 Claude Code 知道這個 skill 叫什麼
- 讓 Claude Code 知道什麼時候要用它
- 用 YAML 格式（不需要懂 YAML）
- 只有 `name` 和 `description` 是必填

進階 Frontmatter 欄位

- `allowed_tools` ：限制可用的工具
- `argument_hint` ：提示參數格式
- `model` ：指定使用的 AI 模型
- `disablePromptCaching` ：控制觸發方式

BEST PRACTICE

保持 `SKILL.md` 在 500 行以內。詳細的 reference 資料放進支援檔案，不要全部塞在 skill 本身。

FILE ARCHITECTURE

支援檔案架構

Skill 不只是一個 md 檔。你可以在旁邊放支援檔案，讓 skill 在需要時去讀取。

OPTION A · SELF-CONTAINED

```
.claude/skills/
  └ transcript/
      ├ SKILL.md
      ├ references/
      └ scripts/
```

支援檔案直接放在 skill 資料夾內。結構清晰，適合獨立的 skill。

OPTION B · EXTERNAL REFERENCES

```
.claude/skills/
  └ transcript/
      └ SKILL.md
references/
  └ patterns.md
scripts/
  └ analyze.py
```

支援檔案放在專案其他位置，SKILL.md 內用路徑指向它們。

KEY RULE

不管哪種架構，只要在 `SKILL.md` 裡用正確路徑指向支援檔案，Claude Code 就能找到它。支援檔案只在需要時才載入，不會每次都讀。

常見支援檔案類型

- **風格規則** ：語氣、用字、格式偏好（如 patterns.md）
- **品牌資料** ：Logo 路徑、色票、字體設定
- **資料檔案** ：頻道數據、競品清單、模板 JSON
- **執行腳本** ：Python / JS script，讓 skill 直接呼叫

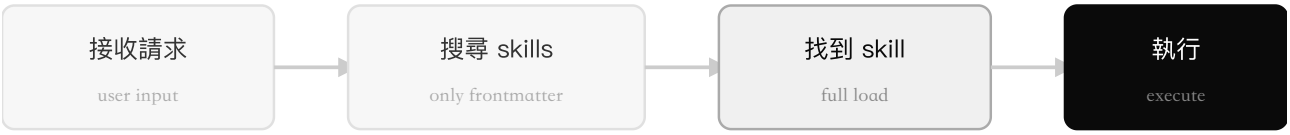
TRIGGERING

觸發方式：Slash Command vs 自然語言

Claude Code 有兩種方式知道要執行哪個 skill。

SLASH COMMAND · 明確觸發	自然語言 · 自動搜尋
`/transcript`	"幫我寫這支影片的腳本"
`/youtube-thumbnail`	"幫我生成 YouTube thumbnail"
`/skill-builder`	"建一個新的 skill"
直接指定 skill 名稱，即時執行，不經過搜尋流程。	Claude Code 自動搜尋最符合的 skill 並執行。

搜尋流程（自然語言觸發時）



NOTE

不是每個請求都會觸發 skill。如果找不到符合的 skill，Claude Code 會直接用一般知識回答。

SYSTEM DESIGN

Progressive Context Loading：3 層架構

Claude Code 不會一次把所有 skill 全讀完。它用 3 層漸進式載入，讓你的 skill 庫再大也不會拖慢速度。

LAYER 1	只讀 Frontmatter	name + description only · 所有 skill 同時掃描	~100 tokens / skill
LAYER 2	讀完整 Skill 指令	找到最符合的 skill 後才完整載入	~1,000 tokens
LAYER 3	載入支援檔案	references · scripts · templates · 按需載入	視需求而定

為什麼這樣設計

- Token 成本：只在需要時才花
- 搜尋效率：Layer 1 掃描成本極低
- 系統層級控制，不靠使用者省

對你的啟示

- **Description 要寫清楚**：Layer 1 靠它找 skill
- **詳細資料放支援檔**：不要塞進 SKILL.md
- **可以有很多個 skill**：不怕 token 爆炸

TIP

Skill description 是 Layer 1 的唯一判斷依據。 如果 skill 不觸發，先回去改 description，讓它更具體地描述觸發時機。

BUILDING

從零建立 Skill：Skill Builder 問答

Skill Builder 是一個 skill，它的工作是幫你建其他 skill。透過問答方式，引導你把需求轉化成完整的 SKILL.md。

01 **你想解決什麼問題？**

描述任務場景，越具體越好。例如：「每次要做 YouTube thumbnail 都很花時間，想一個指令就生成縮圖概念。」

02 **這個 skill 的觸發方式？**

Slash command、自然語言，或兩者都行？有沒有特定的觸發關鍵詞？

03 **從觸發到輸出，步驟是什麼？**

詳細說明流程。例如：接收主題 → 分析風格 → 生成概念 → 輸出圖片。

04 **有沒有支援檔案？Reference 在哪裡？**

品牌設定、風格規則、資料檔案的路徑。Skill Builder 會自動在 SKILL.md 中加入參照。

05 **輸出格式是什麼？**

檔案、純文字、圖片、JSON？要存到哪裡？

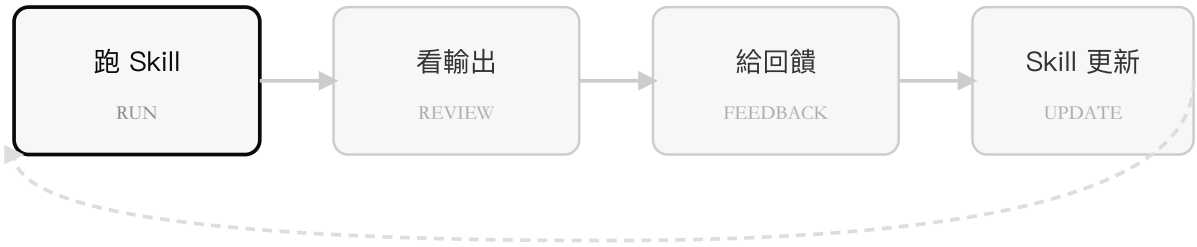
AFTER BUILDER RUNS

Skill Builder 會自動建立 `SKILL.md`、必要的 reference 檔案，並在 `CLAUDE.md` 中註冊這個 skill。接著直接測試它，開始 Feedback Cycle。

ITERATION

Feedback Cycle：邊用邊優化

第一次跑不完美很正常。Skill 是透過不斷迭代變好的，不是一次寫完就結束的。



迭代建議

1 – 2 **前幾次：親眼看著它跑**

不要讓它跑完才看結果，觀察每個步驟，找出可以加速或改進的地方。

5 – 10 **持續給具體回饋**

「輸出格式不對」不夠用。具體說「第三段應該用列表，不是段落文字」。

10+ **Skill 穩定，開始放手**

這時候才能放心讓它背景跑，結果會越來越接近你的期待。

DEBUGGING

Debug 清單：6 種症狀對照表

Skill 跑起來不對勁？對照這張表找到原因。

症狀	解法
步驟順序不對，或跳過某些步驟	→ 修改 `SKILL.md` 指令順序，寫清楚先後關係和依賴條件
輸出的語氣或風格跑掉	→ 建立一個 reference 檔案，把風格規則、語氣範例存進去，讓 skill 去讀
同樣的錯誤一直反覆出現	→ 在 skill 指令裡明確加一條規則：「不可以做 XXX」
一直重複搜尋同樣的資訊	→ 建一個 reference 文件把那個資訊存起來，讓 skill 直接讀，不用每次重新搜
Skill 完全沒有觸發	→ 回去改 frontmatter 的 `description`，寫得更具體，說清楚什麼情況下要用它
Skill 觸發太頻繁，什麼都跑到	→ 在 frontmatter 設定 `disablePromptCaching`，限制為 slash command only

REMEMBER

Skill 不是一次寫完就結束的。它是你跟 Claude Code 一起長期打磨出來的工具。每次修改都讓它更好一點。

SCOPE

Local vs Global Skill

Skill 存在哪裡，決定它的有效範圍。

LOCAL · 專案層級

`.claude/skills/{skill-name}/`

- 只在這個專案資料夾裡有效
- 換到別的專案就不存在
- 適合專案專屬的流程

eg. transcript · yt-title · pdf-resource

GLOBAL · 全域層級

`~/.claude/skills/{skill-name}/`

- 所有專案都能使用
- 不會在專案資料夾中出現
- 適合關於你這個人的 skill

eg. frontend-design · 品牌設定 · 語氣規則

怎麼判斷要裝在哪裡？

裝成 LOCAL 的情況

- 這個 skill 只對這個專案有意義
- 用到的 reference 檔案在這個專案裡
- 你不希望它在其他地方出現

裝成 GLOBAL 的情況

- 這個 skill 描述的是你這個人
- 你的寫作風格、設計語言、品牌設定
- 不管做什麼專案都需要

> SIMPLE RULE
>
> Local 是專案的工具，Global 是你個人的工具。

KEY TAKEAWAYS

三個核心重點

01

Skill = AI 的 SOP，寫一次，反覆用，越用越好

不要每次都重新下指令。把重複的流程寫成 skill，Claude Code 就會記住怎麼做，而且每次迭代都更準確。一個 skill 可以讓你一個人同時跑 4 件事。

02

3 層漸進式載入：skill 庫再大也不會拖慢速度

Layer 1 只讀 frontmatter（~100 tokens），Layer 2 才讀完整指令，Layer 3 按需載入支援檔案。你可以建很多個 skill，不用擔心 token 爆炸。但 description 一定要寫清楚。

03

Feedback Cycle：不是一次到位，是邊用邊優化

第一次跑不完美是正常的。跑 → 看輸出 → 給具體回饋 → skill 更新，重複這個循環。跑個 10 次，skill 就會非常好用。不要寫完就放著，要親眼看著它跑。

ONE MORE THING

官方有 skill 庫，社群也在開源 skill。不需要每個都自己建。但最好用的 skill，永遠是根據你自己的工作流程打磨出來的那一個。

RESOURCES

資源與社群

免費 SKILL 模板

Skool 社群 · 連結在影片說明

Skill Builder · Transcript · Slide Outline · 更多持續新增

YOUTUBE

JayLuxAI | AI 自動化

每週新影片，Claude Code · n8n · AI agent 實戰教學

INSTAGRAM

@jaylux.ai

每日 AI 工具與自動化技巧

— 我是 Jay，我們下支影片見 —